



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Compiladores

Grupo:5CM2

Profra. Albortante Morato Cecilia

Equipo: Bisogno Gandarilla Ricardo

Gutiérrez Espinosa Jordan Gabriel

Ruiz Vazquez Luis Armando

**Trabajo de investigación - Generación de código
intermedio y Optimización de código.**

Índice

Introducción	1
Planteamiento del problema	1
Desarrollo	2
--Diseño de tabla de símbolos	2
--Aplicación	2
--Operaciones	3
--Generación de código intermedio de 3 y 4 direcciones	4
--Generación de código intermedio para estructuras iterativas y de control	4
--Generación de código intermedio para el llamado a funciones y procedimientos	6
-- Introducción a la optimización de código	7
Conclusiones	8
Bibliografía	9

Introducción

La generación de código intermedio y la optimización de código son esenciales en la programación y la compilación de lenguajes de programación. Estas áreas de investigación se centran en el desarrollo de métodos y algoritmos para transformar el código fuente del programa en un formato intermedio que sea más adecuado para su posterior traducción y ejecución eficiente. La generación de código intermedio implica la creación de una representación neutral y estructurada del lenguaje de programación original. Este código intermedio actúa como puente entre el código fuente y el código objeto final. Esto proporciona un mayor nivel de abstracción que facilita el análisis y la optimización posterior.

Por otro lado, la optimización de código se refiere a un conjunto de técnicas que tienen como objetivo mejorar el rendimiento y la eficiencia del código generado. Durante este proceso, el código intermedio se modifica para reducir el tiempo de ejecución, reducir el consumo de recursos y mejorar la legibilidad y el mantenimiento del programa resultante.

El principal objetivo de este trabajo es investigar en profundidad la generación de código intermedio y la optimización de código en el contexto de la compilación de lenguajes de programación. Su objetivo es explorar las últimas técnicas y algoritmos utilizados en estos campos y su aplicación a diferentes paradigmas de programación y plataformas de ejecución.

Planteamiento del problema

Realizar un reporte de investigación acerca de las fases del compilador de generación de código intermedio y optimización de código.

Tratar de abordar la mayoría de los siguientes temas de manera muy general.

- Diseño de tabla de símbolos para soportar llamadas a funciones y/o procedimientos y ámbito de variables.
- Generación de código intermedio de 3 y 4 direcciones.
- Generación de código intermedio para estructuras iterativas y de control (for, while, if-else, switch).
- Generación de código intermedio para el llamado a funciones y procedimientos.
- Introducción a la optimización de código.

Desarrollo

Diseño de tabla de símbolos para soportar llamadas a funciones y/o procedimientos y ámbito de variables.

Una tabla de símbolos es una estructura de datos importante creada y mantenida por el compilador para almacenar información sobre la apariencia de varias entidades, como nombres de variables, nombres de funciones, objetos, clases, interfaces, etc. La tabla de símbolos se utiliza para el análisis y la síntesis del compilador. Dependiendo del lenguaje de señas, la tabla de símbolos se puede utilizar para los siguientes propósitos:

Mantenga todos los nombres de dispositivos en un solo lugar de forma estructurada.

Comprueba si una variable está declarada. Implemente la verificación de tipos para verificar que las asignaciones y expresiones en el código fuente sean semánticamente correctas.

Determinar el alcance (alcance de resolución) del título. Una tabla de símbolos es simplemente una tabla, que puede ser una tabla lineal o una tabla hash.

Mantiene una entrada para cada uno de los nombres con el formato siguiente:

`<symbol name, type, attribute>`

Por ejemplo, si tiene una tabla de símbolos para almacenar información acerca de la siguiente declaración de variables:

```
static int interest;
```

A continuación, debe almacenar la entrada como la siguiente:

`<interest, int, static>`

El atributo cláusula contiene las entradas relacionadas con el nombre.

Aplicación:

Si el compilador trabaja con una pequeña cantidad de datos, es posible implementar la tabla de símbolos como una lista desordenada, que es fácil de codificar, pero solo es adecuada para tablas pequeñas. Una tabla de símbolos se puede implementar de una de las siguientes maneras:

- Lista lineal (ordenada o desordenada).
- Árbol de búsqueda binaria.
- Tabla Hash.

Entre ellos, la tabla de símbolos se implementa principalmente en forma de tabla hash, donde el código fuente del símbolo en sí se considera el elemento principal de la función hash, y el valor de retorno es la información del símbolo.

Operaciones:

Insert():

Esta operación es la que se utiliza con más frecuencia en fase de análisis, es decir, la primera mitad del compilador en donde los testigos son identificados y los nombres se almacenan en la tabla. Esta operación se utiliza para añadir información en la tabla de símbolos de nombres únicos que ocurren en el código fuente. El formato o estructura en la que los nombres se almacenan depende del compilador en mano.

Un atributo de un símbolo en el código fuente es la información asociada con ese símbolo. Esta información contiene el valor, el estado, el alcance y el tipo sobre el símbolo. El insertar() toma el símbolo y sus atributos como argumentos y almacena la información en la tabla de símbolos.

Por ejemplo:

```
int a;
```

El compilador lo procesa como:

```
insert(a, int);
```

Lookup():

Lookup() es utilizado para buscar un nombre en la tabla de símbolos para determinar:

- Si el símbolo existe en la tabla.
- Si se declara antes de que se utilice.
- Si el nombre se usa en el ámbito de aplicación.
- Si el símbolo está inicializado.
- Si el símbolo declarado varias veces.

El formato de Lookup() varía según el lenguaje de programación. El formato básico debe coincidir con el siguiente:

```
Lookup(símbolo)
```

Este método devuelve el valor 0 (cero) si el símbolo no existe en la tabla de símbolos. Si el símbolo existe en la tabla de símbolos, se devuelve sus atributos almacenados en la tabla.

Generación de código intermedio de 3 y 4 direcciones.

La generación de código intermedio de 3 y 4 direcciones es una técnica utilizada durante la compilación de un lenguaje de programación. El código intermedio se genera como una representación simplificada y estructurada del código fuente original, que luego se transforma en el código objeto final.

El código intermedio de 3 direcciones es un formato que utiliza instrucciones con hasta tres operadores. Estas instrucciones se basan principalmente en operaciones aritméticas y lógicas simples como la suma, la resta, la multiplicación y la comparación. Por ejemplo, una instrucción de 3 direcciones podría ser "t1 = a b", donde "t1" es una variable temporal y "a" y "b" son operandos.

La ventaja de la generación de tres direcciones entre códigos es que es fácil de entender y convertir a código de máquina o ensamblador. Además, facilita la aplicación de la optimización y el análisis posterior.

Por otro lado, un código intermedio de 4 direcciones proporciona instrucciones con hasta cuatro operandos. Este formato amplía la funcionalidad del código intermedio de 3 direcciones, lo que permite operaciones más complejas, como llamadas a funciones y acceso a estructuras de datos. 4 La instrucción de dirección puede ser "t1 = a * b c", donde "a", "b" y "c" son operandos. La generación de intercodificación de cuatro vías proporciona una mayor expresividad y flexibilidad en la representación del código fuente original. Sin embargo, dada una instrucción con una gran cantidad de operandos, traducirlos a código de máquina o ensamblador puede ser más complicado. Tanto el código intermedio de 3 direcciones como el de 4 direcciones se utilizan en diferentes etapas del proceso de construcción. Estos formatos actúan como una capa de abstracción entre el código fuente y el código objeto final, lo que permite utilizar la optimización y el análisis para mejorar el rendimiento y la eficiencia del programa final.

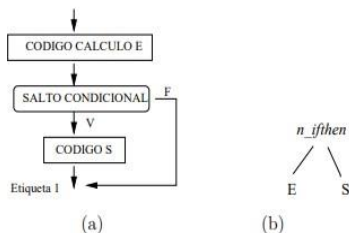
Generación de código intermedio para estructuras iterativas y de control (for, while, if-else, switch).

La generación de código intermedio para iteraciones y estructuras de control como bucles "for" y "while", declaraciones "if-else" y construcciones "switch" es un aspecto esencial del proceso de compilación de un lenguaje de programación. Estas estructuras le permiten controlar el flujo de ejecución del programa y son los elementos principales para implementar algoritmos y lógica condicional. Durante la generación de código intermedio, estas estructuras se convierten en instrucciones y expresiones que representan sus operaciones y comportamiento en un formato más simple y manejable. A continuación se describe cómo generar código intermedio para algunas de estas estructuras.

- Bucles "for" y "while": Los bucles "for" y "while" representan condiciones que determinan si el bloque de código asociado debe continuar. El código intermedio a menudo usa declaraciones de salto condicional y etiquetas para implementar la lógica de control de bucle. Por ejemplo, un ciclo "for" se puede traducir en declaraciones de código intermedio que incluyen comparaciones, saltos condicionales y etiquetas que marcan el inicio del ciclo. si-otra declaración:
- Las declaraciones "If-else" le permiten ejecutar un bloque de código basado en una condición booleana. En el código intermedio, las instrucciones de salto condicional se utilizan para evaluar las condiciones y determinar qué bloque de código ejecutar. Un bloque "si" está asociado con un salto hacia adelante incondicional después de la ejecución, mientras que un bloque "si no" está asociado con un salto hacia adelante incondicional desde el bloque "si" hacia adelante. Estructura del interruptor:
- La construcción "Switch" le permite evaluar expresiones y ejecutar diferentes bloques de código según el valor devuelto. En el código intermedio, la estructura de "cambio" se implementa mediante una combinación de declaraciones de comparación y declaraciones de salto condicional. Cada caso tiene asociada una etiqueta y un salto incondicional hacia adelante o hacia atrás, según la coincidencia entre el valor de la expresión y el valor del registro. Vale la pena señalar que la generación de código intermedio para estas estructuras implica no solo convertirlas en instrucciones apropiadas, sino también el control de variables y el manejo correcto de las ramas de control. Además, se pueden utilizar técnicas de optimización especiales, como la eliminación de código muerto o la optimización de bifurcación condicional, para mejorar la eficiencia del código generado. En resumen, generar código intermedio para iteración y estructuras de control implica convertir estas estructuras en declaraciones y expresiones equivalentes en un formato más simple. Esto permite que el compilador analice y optimice el código de manera más eficiente, asegurando que el flujo de ejecución del programa final sea correcto y eficiente.

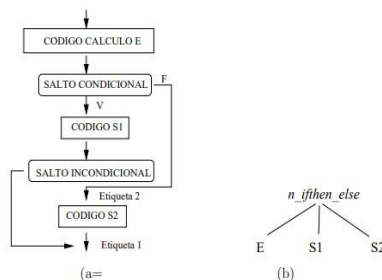
7.6.1. Proposición if-then

Supongamos una sentencia if-then de la forma $S \rightarrow \text{if } E \text{ then } S_1$, ver diagrama de flujo en figura 7.6. Para generar el código correspondiente a esta proposición habría que añadir a la función `genera_código()` un nuevo caso para la sentencia switch que contemple este tipo de nodo en el árbol sintáctico, nodo `n_ifthen`.



7.6.2. Proposición if-then-else

Supongamos una sentencia if-then-else de la forma $S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$, cuyo diagrama de flujo tiene la forma representada en la figura 7.7. Para generar el código correspondiente a esta proposición habría que añadir a la función `genera_código()` un nuevo caso para la sentencia switch que contemple este tipo de nodo en el árbol sintáctico, nodo `n_ifthenelse`. Este fragmento de código se podría implantar de forma similar a como hemos hecho en la sección anterior. Ver ejercicios.



7.6.3. Proposición while-do

Una sentencia while-do tiene la forma $S \rightarrow \text{while } E \text{ do } S_1$, cuyo diagrama de flujo viene representado en la figura 7.8. Para generar el código correspondiente a esta proposición habría que añadir a la función `genera_código()` un nuevo caso para la sentencia switch que contemple este tipo de nodo en el árbol sintáctico.

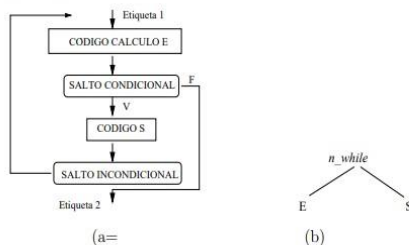


Figura 7.8: (a) Diagrama de flujo para la proposición while-do y (b) tipo de nodo

Generación de código intermedio para el llamado a funciones y procedimientos.

La generación de código intermedio que llama a funciones y procedimientos es una parte importante del proceso de compilación de un lenguaje de programación. Estas llamadas le permiten modularizar su código y reutilizar funciones y procedimientos en diferentes partes de su programa. Durante la generación de carillas, las funciones y los procedimientos de llamada deben realizar las siguientes tareas:

1. Paso de argumentos:

- Los argumentos pasados a una función o procedimiento deben evaluarse y almacenarse en la ubicación correcta antes de la llamada. Para hacer esto, se generan instrucciones que asignan un valor de parámetro a un registro específico o ubicación de memoria de acuerdo con la convención de llamadas del lenguaje.

2. Generación de llamadas:

- a. Genera una instrucción de llamada que especifica una función o procedimiento para ejecutar. La declaración generalmente especifica el nombre o la dirección de la función o procedimiento y, en algunos casos, los parámetros pasados.

3. Acciones de control:

- a. Luego de realizar la llamada, se debe gestionar el código que transfiere el control a la función o procedimiento en cuestión. Esto implica crear un nuevo punto de entrada para la ejecución y almacenar una dirección de retorno que le permita volver al punto de llamada después de que se haya completado la función o el procedimiento.

4. Retorno de valores:

- a. Si una función o procedimiento devuelve un valor, se deben generar instrucciones para almacenar el valor en una ubicación adecuada, como un registro o una variable local. En algunos casos, es posible que se requieran instrucciones adicionales para señalar el final de la ejecución de la función o el procedimiento y regresar al punto de llamada.

Es importante señalar que el manejo de parámetros y la transferencia de control pueden variar según el método de invocación utilizado por el lenguaje de programación y la arquitectura de la plataforma de tiempo de ejecución. Algunos métodos de llamada populares implican pasar argumentos a través de registros, la pila o una combinación de ambos.

Introducción a la optimización de código.

La optimización de código es un conjunto de fases del compilador que transforma un fragmento de código en otro que funciona de manera equivalente y que funciona de manera más eficiente, es decir utiliza menos recursos informáticos (como la memoria o el tiempo de ejecución).

Es importante decir que:

La condición "comportamiento equivalente" es bastante engorrosa, ya que también incluye casos de error en los que el comportamiento debería ser el mismo. Por ejemplo, usamos instrucciones como " $x = y / y$ ". Puede sentirse tentado a reemplazar " $x = 1$ " con esta expresión, pero debe asegurarse de que la variable y

no pueda ser igual a 0, porque entonces el código puede comportarse de manera diferente según el idioma (como dividir por cero).

También es importante asegurarse de que el código no sea menos eficiente que antes de la optimización. Por ejemplo, se puede desenrollar un bucle simplificando instrucciones innecesarias, p. lúpulo) aunque esto aumenta el tamaño del código. Debido a que el tamaño del código puede afectar el nivel de caché y el acceso a la memoria, el programa optimizado puede ejecutarse más lentamente que el programa original.

Existen diferentes técnicas para optimizar el código, cada una de las cuales intenta mejorar un aspecto diferenciado del código. En general pueden clasificarse en dos categorías, las de flujo de datos (data-flow) y las de flujo de control (control-flow). Las optimizaciones del flujo de datos pretenden mejorar la eficiencia de los diferentes cálculos realizados en el programa: precalculando expresiones con valor conocido en tiempo de compilación, reaprovechando cálculos ya realizados en otras partes del código o suprimiendo cálculos innecesarios, ... Por contra, las optimizaciones del flujo de control intentan utilizar las instrucciones de salto condicional e incondicional de la forma más eficiente posible (ya sea desplazando código o eliminando saltos innecesarios). Puede definirse también una tercera categoría, las optimizaciones de bucles (loop optimization), intenta mejorar el rendimiento de las instrucciones iterativas como for, while ... do o repeat ... until. En este caso, los cambios realizados al código del bucle afectan tanto al flujo de datos como al flujo de control.

Conclusiones

En conclusión, la generación de código intermedio y la optimización de código desempeñan un papel crucial en el proceso de compilación de lenguajes de programación. Estas áreas de estudio se centran en transformar el código fuente en una representación intermedia estructurada y mejorar su eficiencia y rendimiento.

Durante la generación de código intermedio, se simplifica y abstrae el código fuente original en un formato más manejable, lo que facilita el análisis, la optimización y la traducción posterior a código objeto o ensamblador. Los formatos de código intermedio de 3 y 4 direcciones se utilizan comúnmente, ofreciendo diferentes niveles de expresividad y complejidad.

Además, la generación de código intermedio abarca la implementación de estructuras iterativas y de control, como bucles "for" y "while", sentencias "if-else" y estructuras "switch". Estas estructuras se traducen en instrucciones de salto condicional y etiquetas que controlan el flujo de ejecución del programa.

Por otro lado, la optimización de código tiene como objetivo mejorar el rendimiento y la eficiencia del programa resultante. Se aplican diversas técnicas, como la eliminación de código muerto, la optimización de saltos condicionales y la

propagación de constantes, para reducir el tiempo de ejecución, minimizar el consumo de recursos y mejorar la legibilidad y mantenibilidad del código generado.

En conjunto, la generación de código intermedio y la optimización de código contribuyen al desarrollo de compiladores más eficientes y programas de software de alto rendimiento.

Bibliografía

- 1.- Compilador Diseño - Tabla de Símbolos. (s/f). Tutorialspoint.com. Recuperado el 25 de junio de 2023, de https://www.tutorialspoint.com/es/compiler_design/compiler_design_symbol_table.html
- 2.- Wikipedia contributors. (s/f). *Código de tres direcciones*. Wikipedia, The Free Encyclopedia. https://es.wikipedia.org/w/index.php?title=C%C3%B3digo_de_tres_direcciones&oldid=125249164
- 3.- *Capítulo 7 Generación de código intermedio. Optimización*. (s/f). Informatica.uv.es. Recuperado el 25 de junio de 2023, de <http://informatica.uv.es/docencia/iiguia/asignatu/2000/PL/2008/tema7.pdf>
- 4.- (S/f). Edu.ar. Recuperado el 25 de junio de 2023, de <https://rdu.unc.edu.ar/bitstream/handle/11086/41/15762.pdf?sequence=1>
- 5.- Viladrosa, R. C. (2016, mayo 2). *¿Qué hay que tener en cuenta en la optimización de código?* Tecnología++. <https://blogs.uoc.edu/informatica/optimizacion-de-codigo-un-codigo-mas-eficiente/>